TP3

Dash fundamentals

MECD 2024/25 **Advanced Data Visualization**

Evgheni Polisciuc

1290 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

dei25 departamento de engenharia informática 1995 – 2020

Outline

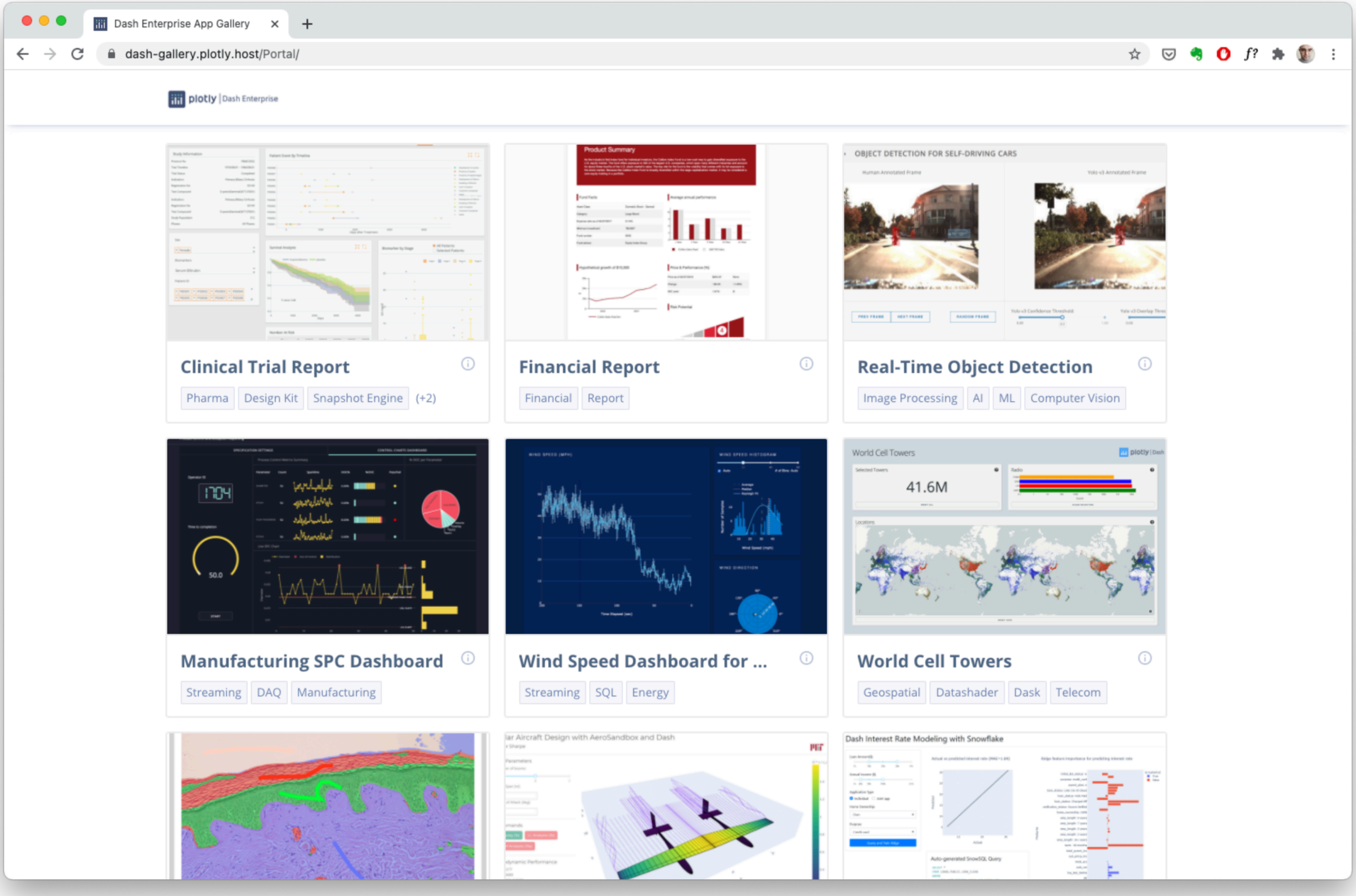
1. **Introduction**
2. Layout
3. Callbacks

Introduction

- Dash is a framework for building highly interactive web analytic applications;
- Built on top of Flask, Plotly.js, and React.js;
- Allows creating web-based applications using Python;
- Dash apps are rendered in browsers, so can be deployed to servers;
- Dash apps are composed of two parts: i) the layout, ii) the interactivity;
- Based on the 3 pillars: i) components; ii) graphs; and iii) callbacks.

Introduction

https://dash-gallery.plotly.host/Portal/



Introduction

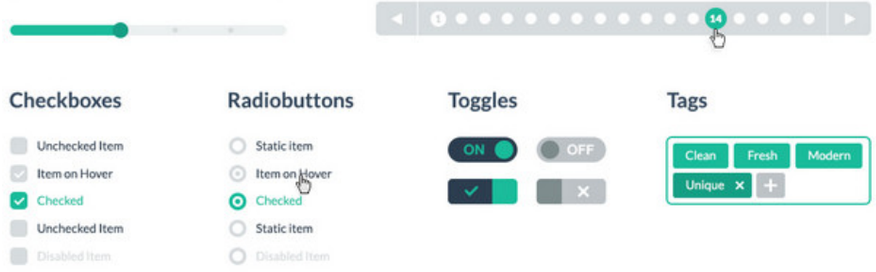
Installation

- Dash requires its own installation: `pip install dash`
- This installs all the three component libraries that form the core Dash: `html, core, table`
- In order to use Dash with Jupiter Lab follow this instructions: https://dash.plotly.com/workspaces/using-dash-in-jupyter-and-workspaces

Introduction

Two parts, 3 pillars

- Part 1: the **Layout**
  - Pillar 1: Dash **components**
    - Core components
    - HTML components
  - Pillar 2: Plotly **graphs**
    - Graphs are also Dash components
    - dcc.Graph(figure=fig) with fig a plotly figure
- Part 2: the **Interaction**
  - Pillar 3: Callbacks
    - Link the dash components: components + graphs



Introduction

Basic example

Layout

```python
from dash import Dash, dcc, html
from dash.dependencies import Input, Output
import plotly.graph_objects as go

app = Dash(__name__)

app.layout = html.Div([
    html.P("Color:"),
    dcc.Dropdown(
        id="dropdown",
        options=[
            {'label': x, 'value': x}
            for x in ['Gold', 'MediumTurquoise',
'LightGreen']
        ],
        value='Gold',
        clearable=False,
    ),
    dcc.Graph(id="graph"),
])
```

Interaction

```python
@app.callback(
    Output("graph", "figure"),
    [Input("dropdown", "value")])
def display_color(color):
    print(color)
    fig = go.Figure(
        data=go.Bar(y=[2, 3, 1], marker_color=color))
    return fig

if __name__ == "__main__":
    app.run_server(debug=True)
```

Outline

1. Introduction
2. **Layout**
3. Callbacks

Dash layout

Components

- Dash provides Python classes for all of the visual components;
- These components are contained in two libraries:
  - core components
    - High-level components (generated with JavaScript, HTML, CSS through React.js);
    - Usually imported as dcc;
    - See documentation here (https://dash.plotly.com/dash-core-components).
  - html components
    - Python structures for all of HTML tags;
    - Usually imported as html;
    - The style properties is a dictionary;
    - The class properties is named as className
    - See documentation here (https://dash.plotly.com/dash-html-components)

Dash layout

Example

```
…

app.layout = html.Div(children=[
    html.H1(children='Hello Dash'),

    html.Div(children='''
        Maritime empires: Some of the maritime empires. Choose one.
    '''),

    dcc.Dropdown(
        id='demo-dropdown',
        options=[
            {'label': 'Portugal', 'value': 'PT'},
            {'label': 'Spain', 'value': 'ES'},
            {'label': 'France', 'value': 'FR'}
        ],
        value='PT'
    )
])

…
```

- **Layout**: composed of a tree structure: div. which contains h1, div html tags and a dropdown dash component;
- Html.H1(children=‘Hello Dash’) generates <h1>Hello Dash</h1> html elements;
- The children keyword is special and can be omitted if in first place.

Dash layout

Css styling

```
external_stylesheets = ['css/myStyle.css']

app = Dash(__name__, external_stylesheets=external_stylesheets)
```

- You can use external css styles to customize the visual appearance of your app.
- Use external_stylesheets property for that

Dash layout

Css styling

```
html.H1(
    children='Hello Dash',
    style={
        'textAlign': 'center',
        'color': '#7FDBFF'
    }
)
```

- Inline styles can be modified using the style property

Dash layout

Reusable components

```
df = pd.read_csv('https://gist.githubusercontent.com/chriddyp/
c78bf172206ce24f77d6363a2d754b59/raw/
c353e8ef842413cae56ae3920b8fd78468aa4cb2/usa-agricultural-exports-2011.csv')


def generate_table(dataframe, max_rows=10):
    return html.Table([
        html.Thead(
            html.Tr([html.Th(col) for col in dataframe.columns])
        ),
        html.Tbody([
            html.Tr([
                html.Td(dataframe.iloc[i][col]) for col in dataframe.columns
            ]) for i in range(min(len(dataframe), max_rows))
        ])
    ])


external_stylesheets = ['https://codepen.io/chriddyp/pen/bWLwgP.css']

app = Dash(__name__, external_stylesheets=external_stylesheets)

app.layout = html.Div(children=[
    html.H4(children='US Agriculture Exports (2011)'),
    generate_table(df)
])

if __name__ == '__main__':
    app.run_server(debug=True)
```

- Don’t forget that you can create complex reusable components using Python functions

Dash layout

Graph component

- On of the most important core components is the dcc.Graph component;
- Renders interactive visualizations using Plotly graph objects;
- The figure attribute holds the graph object;

```
app = Dash(__name__)

fig = go.Figure(data=
    go.Scatterpolar(
        r = [0.5,1,2,2.5,3,4],
        theta = [35,70,120,155,205,240],
        mode = 'markers',
    ))

fig.update_layout(showlegend=False)

app.layout = html.Div([
    dcc.Graph(id="graph", figure=fig)
])

if __name__ == "__main__":
    app.run_server(debug=True)
```

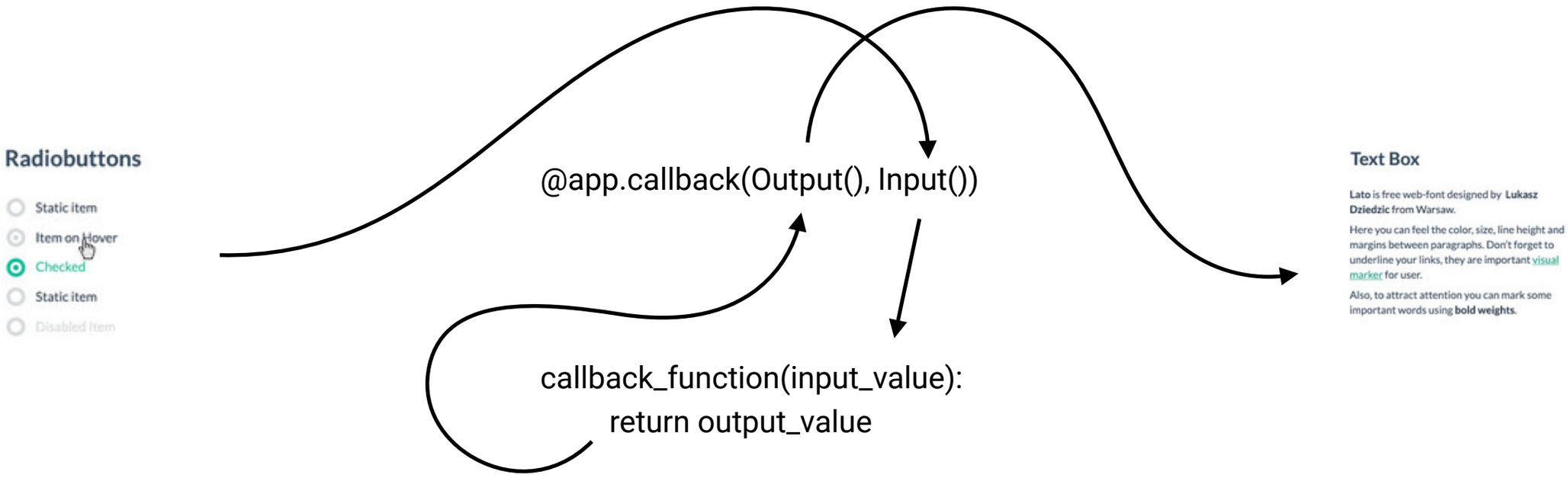
Outline

1. Introduction
2. Layout
3. **Callbacks**

Dash Interactivity

Simple callbacks

- Dash callbacks are Python functions that are called whenever an input component's property changes. E.g., select different year in the dropdown menu;
- Dash uses '@app.callback' decorator (revise or learn about decorators here);
- The communication between components though the callback is made using the 'inputs' and 'outputs' that are declared as the arguments of the decorator;



Dash Interactivity

Example

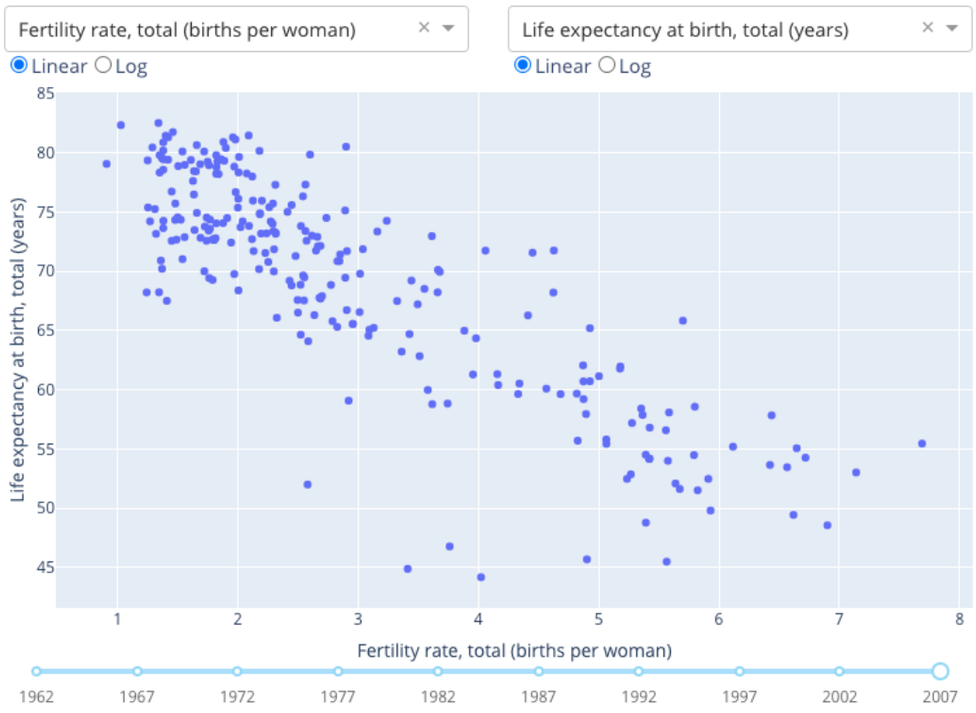
```
from dash.dependencies import Input, Output

app = Dash(__name__)

app.layout = html.Div([
    html.H6("Change the value in the text box to see callbacks in action!"),
    html.Div(["Input: ",
              dcc.Input(id='my-input', value='initial value', type='text')]),
    html.Br(),
    html.Div(id='my-output'),

])


@app.callback(
    Output(component_id='my-output', component_property='children'),
    Input(component_id='my-input', component_property='value')
)
def update_output_div(input_value):
    return 'Output: {}'.format(input_value)


if __name__ == '__main__':
    app.run_server(debug=True)
```

- The ‘input’ and ‘output’ declares the components that are interact;
- The linkage is made though the ids of the components;
- The values for both input and the output arguments are passes thought the use of component’s properties;
- The keywords component_id and component_property are optional;
- The callback function is declared right after the decorator (no blank line can appear in-between).

Dash Interactivity

Multiple inputs

```
…
@app.callback(
    Output('indicator-graphic', 'figure'),
    Input('dropdown_xaxis', 'value'),
    Input('dropdown_yaxis', 'value'),
    Input('radio_xaxis-type', 'value'),
    Input('radio_yaxis-type', 'value'),
    Input('year--slider', 'value'))
def update_graph(xaxis_column_name, yaxis_column_name,
                 xaxis_type, yaxis_type,
                 year_value):
    dff = df[df['Year'] == year_value]

    x=dff[dff['Indicator Name'] == xaxis_column_name]['Value']
    y=dff[dff['Indicator Name'] == yaxis_column_name]['Value']
    hover_name=dff[dff['Indicator Name'] == yaxis_column_name]['Country
Name']

    …

    fig.update_xaxes(title=xaxis_column_name,
                     type='linear' if xaxis_type == 'Linear' else 'log')

    fig.update_yaxes(title=yaxis_column_name,
                     type='linear' if yaxis_type == 'Linear' else 'log')

    return fig
…
```



- Any output can have multiple input components

Dash Interactivity

Multiple outputs

```
…
@app.callback(
    Output('square', 'children'),
    Output('cube', 'children'),
    Output('twos', 'children'),
    Output('threes', 'children'),
    Output('x^x', 'children'),
    Input('num-multi', 'value'))
def callback_a(x):
    return x**2, x**3, 2**x, 3**x, x**x
…
```

- Several outputs can be update at once

Dash Interactivity

States

```
…
@app.callback(Output('output-state', 'children'),
              Input('submit-button-state', 'n_clicks'),
              State('input-1-state', 'value'),
              State('input-2-state', 'value'))
def update_output(n_clicks, input1, input2):
    return u'''
        The Button has been pressed {} times,
        Input 1 is "{}",
        and Input 2 is "{}"
    '''.format(n_clicks, input1, input2)
…
```

- In some cases it might be necessary to pass extra values to the callback without firing it;
- In this situations we use State object.

Questions?

References

- Documentation:
  - Core dash components: https://dash.plotly.com/dash-core-components
  - HTML dash components: https://dash.plotly.com/dash-html-components
  - Dash Graph component: https://dash.plotly.com/dash-core-components/graph
- Gallery of Dash apps: https://dash-gallery.plotly.host/Portal/
- https://dash.plotly.com/introduction